VIGILADA MINEDUCACIÓN

UNIVERSIDAD DEL QUINDÍO
Res.MEN 014915 - 02 AGO 2022
RENOVACIÓN ACREDITACIÓN

Algorit Finance

Software Architecture Description – C4 Model

Autor:
Sarita Londoño Perdomo
Eyner Andres Diaz

Versión: 1.0
Fecha: 25/04/2026

UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

1. RESUMEN ARQUITECTÓNICO

FoodExpress UQ es una aplicación de pedidos de comida en línea cuyo propósito es centralizar y automatizar el ciclo completo del pedido (creación, pago, asignación, seguimiento y entrega) mediante canales digitales diferenciados por rol. El cliente realiza pedidos y consulta su historial desde una aplicación móvil; el repartidor gestiona entregas y actualiza estados desde una aplicación móvil; y el administrador supervisa y controla la operación mediante una aplicación web. El sistema se integra con una pasarela de pago externa para procesar transacciones y con un sistema de notificaciones para informar eventos relevantes durante el ciclo del pedido.

1.1 Usuarios principales

- **Cliente:** usa app móvil para explorar, pedir, pagar, consultar historial y ver tiempo estimado de entrega.
- **Repartidor:** usa app móvil para ver pedidos asignados, aceptar/confirmar, actualizar estados y registrar entrega.
- **Administrador:** usa web para gestionar pedidos, usuarios, repartidores, restaurantes/menús y reportes.

1.2 Problema que resuelve

El sistema resuelve la necesidad de gestionar pedidos de comida con trazabilidad completa, reduciendo la coordinación manual entre cliente, operación y reparto. Permite seguimiento del pedido y su tiempo estimado de entrega, integra pagos electrónicos seguros mediante un tercero y automatiza notificaciones sobre cambios de estado para mejorar la experiencia del usuario y el control operativo.

1.3 Alcance

¿Qué incluye?

- Registro y autenticación de clientes, repartidores y administradores.
- Realización de pedidos de comida desde la aplicación móvil del cliente.
- Consulta del historial de pedidos y tiempos estimados de entrega.
- Gestión y asignación de pedidos a repartidores.
- Actualización del estado del pedido durante el proceso de entrega.
- Administración del sistema desde una aplicación web.
- Integración con una pasarela de pago externa.
- Seguimiento del pedido y estimación de tiempo de entrega (por estados y eventos del proceso).
- Envío de notificaciones automáticas a usuarios.

¿Qué **NO** incluye?

- Preparación física de los alimentos por parte de los restaurantes.
- Gestión interna de inventarios de los restaurantes.
- Optimización avanzada de rutas mediante sistemas de geolocalización externos.

1.4 Procesos involucrados en el sistema FoodExpress UQ

- **Proceso de gestión de usuarios**
  Objetivo: permitir el acceso controlado al sistema según el rol.
  Incluye:
  - Registro de usuarios (cliente y repartidor).
  - Autenticación e inicio de sesión.
  - Asignación de roles (cliente, repartidor, administrador).
  - Gestión de perfiles de usuario.

- **Proceso de gestión de pedidos**
  Objetivo: permitir que el cliente solicite comida desde la aplicación móvil.
  Incluye:
  - Selección de productos de comida.
  - Creación del pedido.
  - Confirmación del pedido.
  - Envío del pedido al sistema central.

  Objetivo: controlar el ciclo de vida del pedido.
  Incluye:
  - Registro del pedido en el sistema.
  - Cambio de estados (creado, pagado, en preparación, en entrega, entregado).
  - Consulta del estado del pedido por parte del cliente.

  Objetivo: permitir la trazabilidad de los pedidos.
  Incluye:
  - Consulta del historial de pedidos por parte del cliente.
  - Visualización de tiempos estimados y estados.
  - Consulta de pedidos entregados por el repartidor.

- **Proceso de gestión repartidor**
  Objetivo: asignar un repartidor disponible al pedido.
  Incluye:
  - Identificación de repartidores disponibles.
  - Asignación del pedido.
  - Notificación al repartidor asignado.

  Objetivo: garantizar la entrega del pedido al cliente.
  Incluye:
  - Consulta del pedido por el repartidor.
  - Actualización del estado durante la entrega.
  - Confirmación de entrega.

- **Proceso de administración del sistema**
  Objetivo: permitir la supervisión y control operativo.
  Incluye:
  - Gestión de usuarios.
  - Gestión de pedidos.
  - Gestión de repartidores.
  - Visualización de reportes y estadísticas.

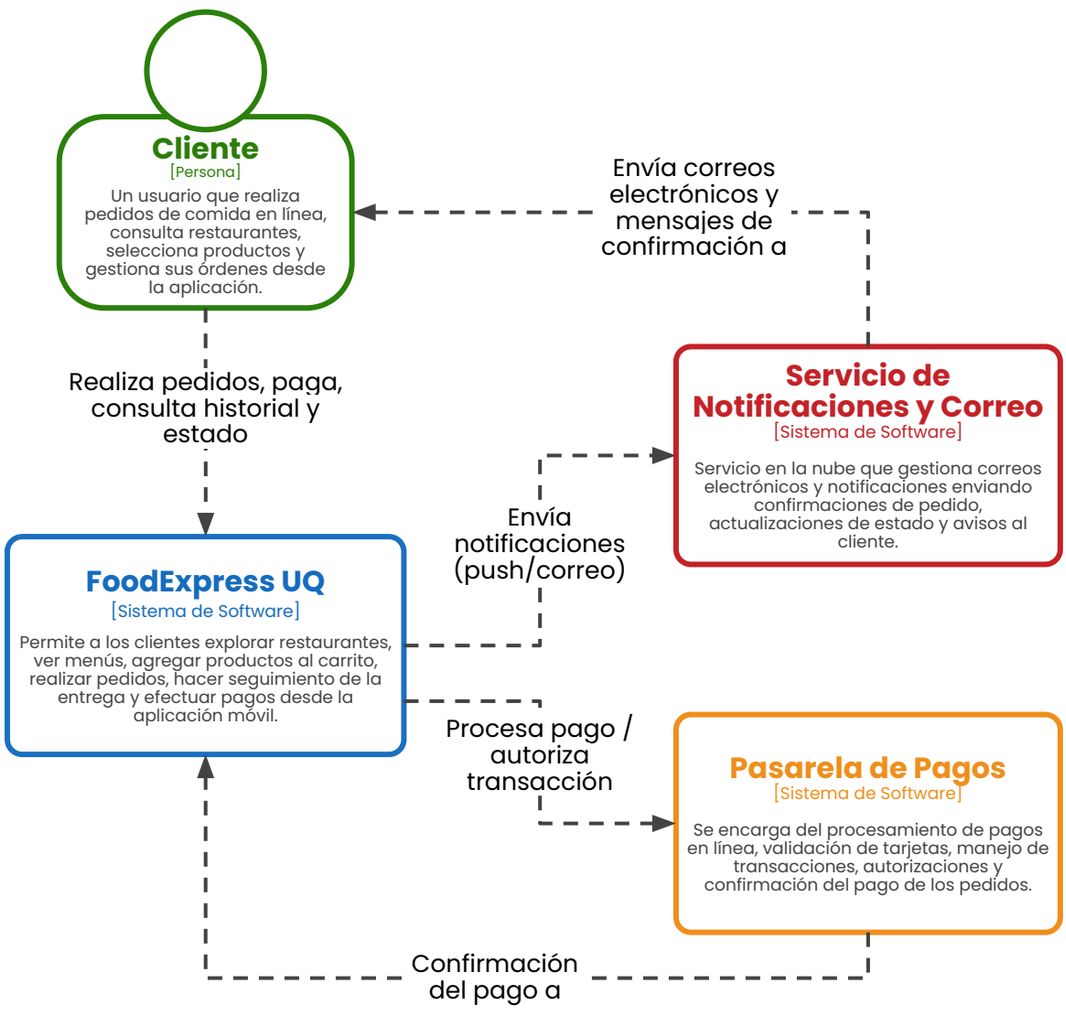
2. C4MODEL

Aunque **FoodExpress UQ** contempla los roles de cliente, repartidor y administrador, las vistas C4 presentadas en este documento se enfocan únicamente en el alcance del cliente (app móvil, backend, base de datos e integraciones externas). Las capacidades de repartidor y administrador forman parte del sistema global descrito en el Ítem 1, pero se documentarán en futuras versiones.

2.1 Diagrama de contexto (N1)

Presenta el sistema FoodExpress UQ como "caja negra", usuarios y sistemas externos.

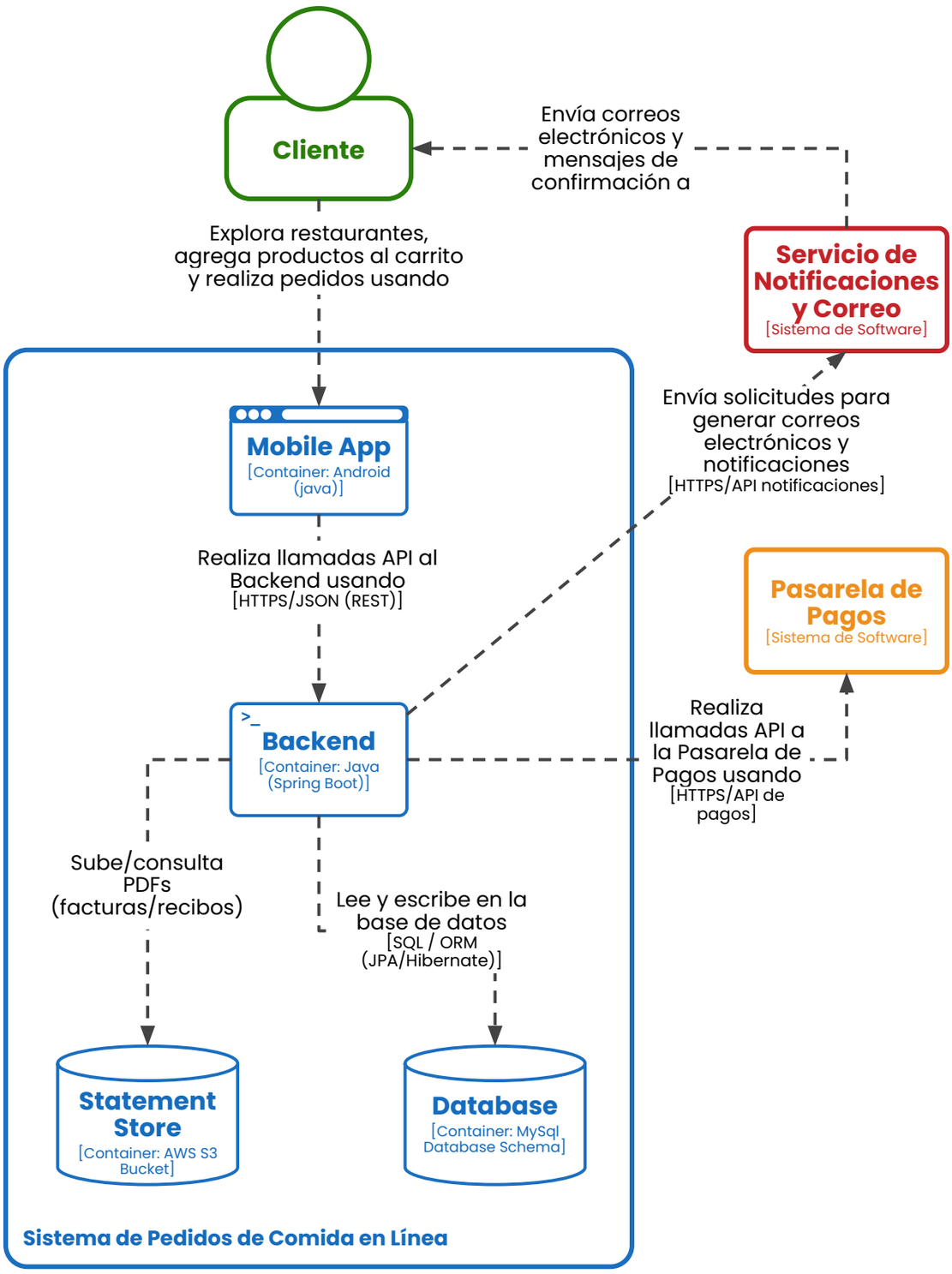
- **Cliente**



2.2 Diagrama de contenedores (N2)

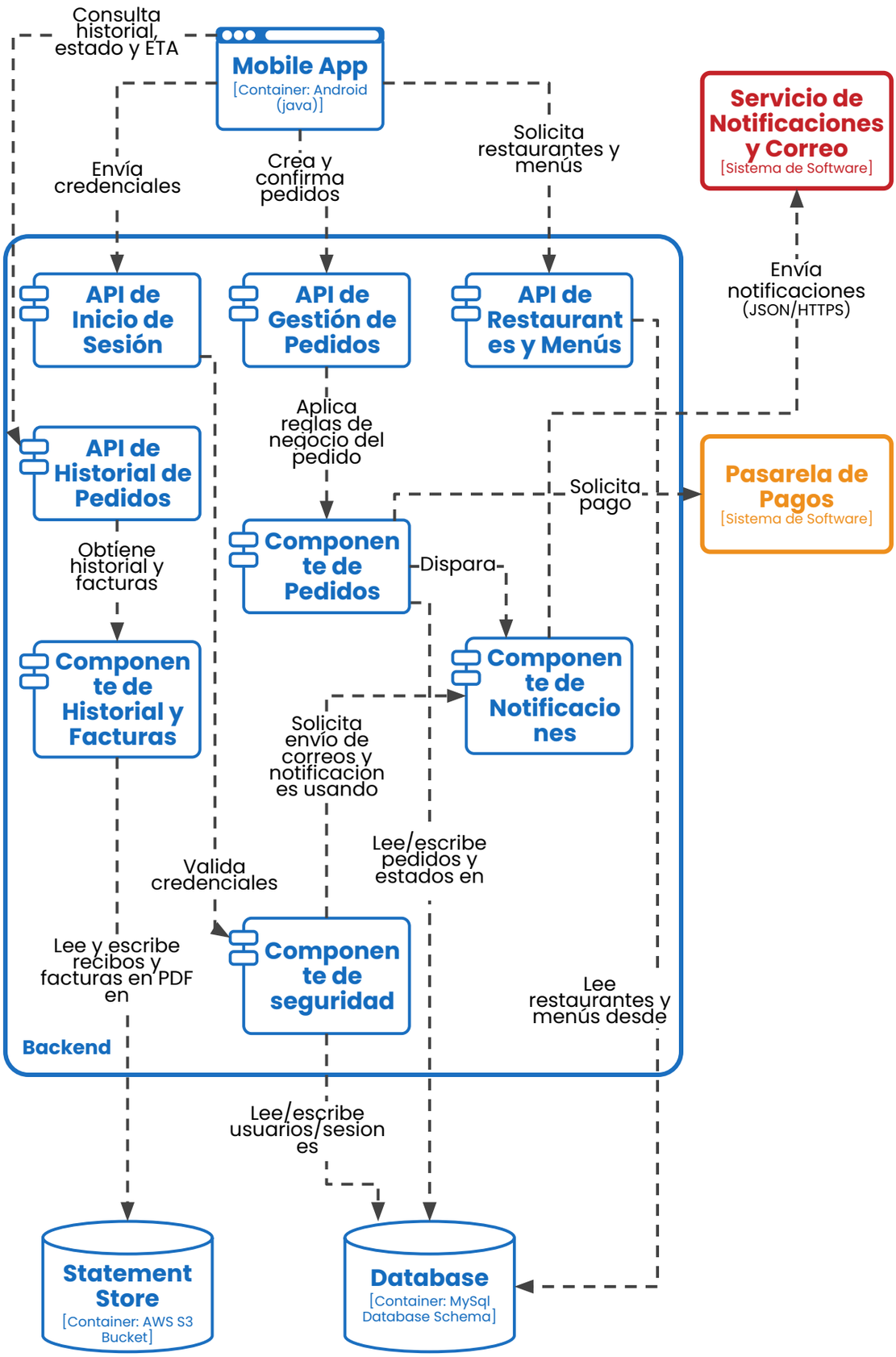
Presenta "qué partes ejecutables" tiene el sistema FoodExpress UQ (web, API, BD, app móvil, colas, etc.).

- **Cliente**



2.3 Diagrama de componentes (N3)

Presenta el interior de los contenedor de FoodExpress (por ejemplo, el Backend/API).



Nota: En la arquitectura del sistema se distingue entre la API de Pedidos, que actúa como el punto de entrada exponiendo los servicios REST consumidos por la aplicación móvil, y el Componente de Pedidos, que concentra la lógica de negocio, la gestión de estados y la coordinación con otros componentes del sistema.

2.4 Código (N4)

2.4.1 Estructura del repositorio y módulos

El sistema se implementa en dos módulos principales alineados con el Nivel 2 (Contenedores): una aplicación móvil Android (Java) como canal del cliente y un Backend en Java (Spring Boot) que centraliza la lógica de negocio e integraciones externas.

Estructura (árbol):

```
/foodexpress-uq/
  /mobile-app-android/
      /app/
          /src/main/
              /java/com/foodexpress/client/
                  /ui/                (Activities/Fragments)
                  /viewmodel/         (lógica de presentación)
                  /network/           (API client, requests)
                  /model/             (DTOs y modelos)
                  /util/              (helpers, constantes)

              /res/
                  /layout/            (XML layouts)
                  /drawable/
                  /values/
                  /mipmap/

          build.gradle


/backend/
  /src/main/java/com/foodexpress/api/
      /controller/     (APIs REST: Auth, Orders, History, Restaurants)
      /service/        (reglas de negocio: AuthService, OrderService, HistoryService)
      /repository/     (acceso a datos: UserRepository, OrderRepository)
      /model/          (Entities JPA: User, Order, OrderStatus, etc.)
      /integration/    (PaymentGatewayAdapter, NotificationClient, StatementStoreClient*)
      /config/         (SecurityConfig*, configuración general)
  /src/main/resources/
      application.yml
      /db/migration*       (Flyway/Liquibase si aplica)
  pom.xml (o build.gradle)

/docs/
  C4Model/
  arquitectura/
  decisiones-arquitectonicas/
```

2.4 Código (N4)

2.4.2 Trazabilidad paquetes/Clases

Para garantizar consistencia entre arquitectura y código, los componentes del Nivel 3 se evidencian en los siguientes módulos:

Elemento N3	Código (paquete/clase)	Responsabilidad
API de Inicio de Sesión	...controller.AuthController	Expone endpoints de autenticación.
Componente de Seguridad	...service.AuthService + ...config.SecurityConfig	Valida credenciales, gestiona sesión/token y protege endpoints.
API de Restaurantes y Menús	...controller.RestaurantController	Expone consulta de restaurantes y menús.
API de Gestión de Pedidos	...controller.OrderController	Expone creación/confirmación de pedidos.
Componente de Pedidos	...service.OrderService	Reglas del pedido, estados, coordinación de pago y notificaciones.
API de Historial de Pedidos	...controller.HistoryController	Expone historial, estado y tiempo estimado del pedido.
Componente de Historial y Facturas	...service.HistoryService	Consulta historial y gestiona recibos/facturas.
Componente de Notificaciones	...service.NotificationService	Dispara y gestiona notificaciones al cliente.
Adaptador de Pasarela de Pagos	...integration.PaymentGatewayAdapter	Encapsula integración con pasarela externa.
Persistencia de Usuarios	...repository.UserRepository	CRUD y consultas de usuarios/sesiones.
Persistencia de Pedidos	...repository.OrderRepository	CRUD y consultas de pedidos/estados.
Persistencia de Historial	...repository.OrderRepository *(o HistoryRepository si existe)*	Consulta histórica de pedidos y estados.
Statement Store (PDFs) *(si aplica)*	...integration.StatementStoreClient	Manejo de recibos/facturas en PDF (S3 u objeto).

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-01: Canal del cliente mediante aplicación móvil Android

- **Decisión:** Implementar el canal del cliente como una aplicación móvil Android desarrollada en Java.
- **Contexto:** El cliente requiere una experiencia móvil para realizar pedidos, consultar historial y recibir notificaciones.
- **Alternativas:** Aplicación web responsive; app híbrida.
- **Justificación:** Permite una experiencia móvil nativa y control del ciclo de vida de la interfaz del cliente.
- **Consecuencias:** Se gestionan versiones y despliegues propios de Android.

ADR-02: Backend centralizado como API REST

- **Decisión:** Implementar un Backend API independiente, expuesto como servicios REST sobre HTTPS.
- **Contexto:** La app móvil debe consumir funcionalidades de autenticación, pedidos, historial, pagos y notificaciones.
- **Alternativas:** Backend monolítico con UI server-side; GraphQL; gRPC.
- **Justificación:** REST es simple, ampliamente soportado y facilita la separación cliente/servidor.
- **Consecuencias:** Se deben versionar endpoints, documentar contratos y manejar errores de red.

ADR-03: Tecnología del backend: Java + Spring Boot

- **Decisión:** Desarrollar el backend en Java utilizando Spring Boot.
- **Contexto:** Se requiere un framework maduro para construir APIs, integrar seguridad y conectarse con BD e integraciones externas.
- **Alternativas:** Node.js/Express; .NET; Django.
- **Justificación:** Spring Boot ofrece un ecosistema estable y productivo para APIs empresariales.
- **Consecuencias:** Mayor estructura y configuración que soluciones más ligeras.

ADR-04: Persistencia transaccional en MySQL

- **Decisión:** Usar MySQL como base de datos principal para usuarios, pedidos, estados e historial.
- **Contexto:** Se manejan entidades transaccionales con relaciones y necesidad de consistencia.
- **Alternativas:** PostgreSQL; NoSQL (MongoDB); archivos.
- **Justificación:** Modelo relacional favorece integridad y consultas confiables para historial y trazabilidad.
- **Consecuencias:** Requiere diseño de esquema e índices para desempeño.

3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-05: Acceso a datos mediante repositorios y ORM

- **Decisión:** Encapsular acceso a datos en una capa de Repository usando ORM (por ejemplo JPA/Hibernate).
- **Contexto:** Evitar acceso directo a la BD desde controladores y mantener separación por responsabilidades.
- **Alternativas:** SQL directo en controladores; DAOs sin ORM.
- **Justificación:** Mejora mantenibilidad, pruebas y consistencia de acceso a datos.
- **Consecuencias:** Curva de aprendizaje y necesidad de ajustar consultas para rendimiento.

ADR-06: Integración de pagos a través de un adaptador en el backend

- **Decisión:** Integrar la Pasarela de Pago desde el backend mediante un PaymentGatewayAdapter.
- **Contexto:** Los pagos son un sistema externo y deben gestionarse de forma segura y centralizada.
- **Alternativas:** Integrar pagos desde la app móvil; acoplar lógica de pagos directamente en controladores.
- **Justificación:** Reduce acoplamiento al proveedor, centraliza validaciones y evita exponer detalles sensibles en el cliente.
- **Consecuencias:** Dependencia del servicio externo; se deben manejar timeouts y fallos.

ADR-07: Notificaciones externalizadas con servicio dedicado

- **Decisión:** Centralizar el envío de notificaciones en un NotificationService que consume un servicio externo.
- **Contexto:** Se debe informar al cliente sobre eventos del pedido (confirmación, pago, cambios de estado).
- **Alternativas:** Enviar notificaciones desde múltiples módulos; notificar solo desde la app.
- **Justificación:** Unifica la lógica de notificación y facilita cambiar de proveedor sin reescribir el sistema.
- **Consecuencias:** Necesidad de manejo de reintentos y registro de fallos.

ADR-08: Manejo de facturas/recibos en almacenamiento de objetos (si aplica)

- **Decisión:** Guardar recibos/facturas PDF en un Statement Store (p.ej., S3) y mantener en MySQL solo la referencia.
- **Contexto:** Archivos PDF no deben almacenarse como blobs en la BD por tamaño y costos.
- **Alternativas:** Guardar PDFs en BD; almacenamiento local del servidor.
- **Justificación:** Escala mejor y reduce carga sobre la BD.
- **Consecuencias:** Se debe controlar acceso a archivos (URLs firmadas o permisos).

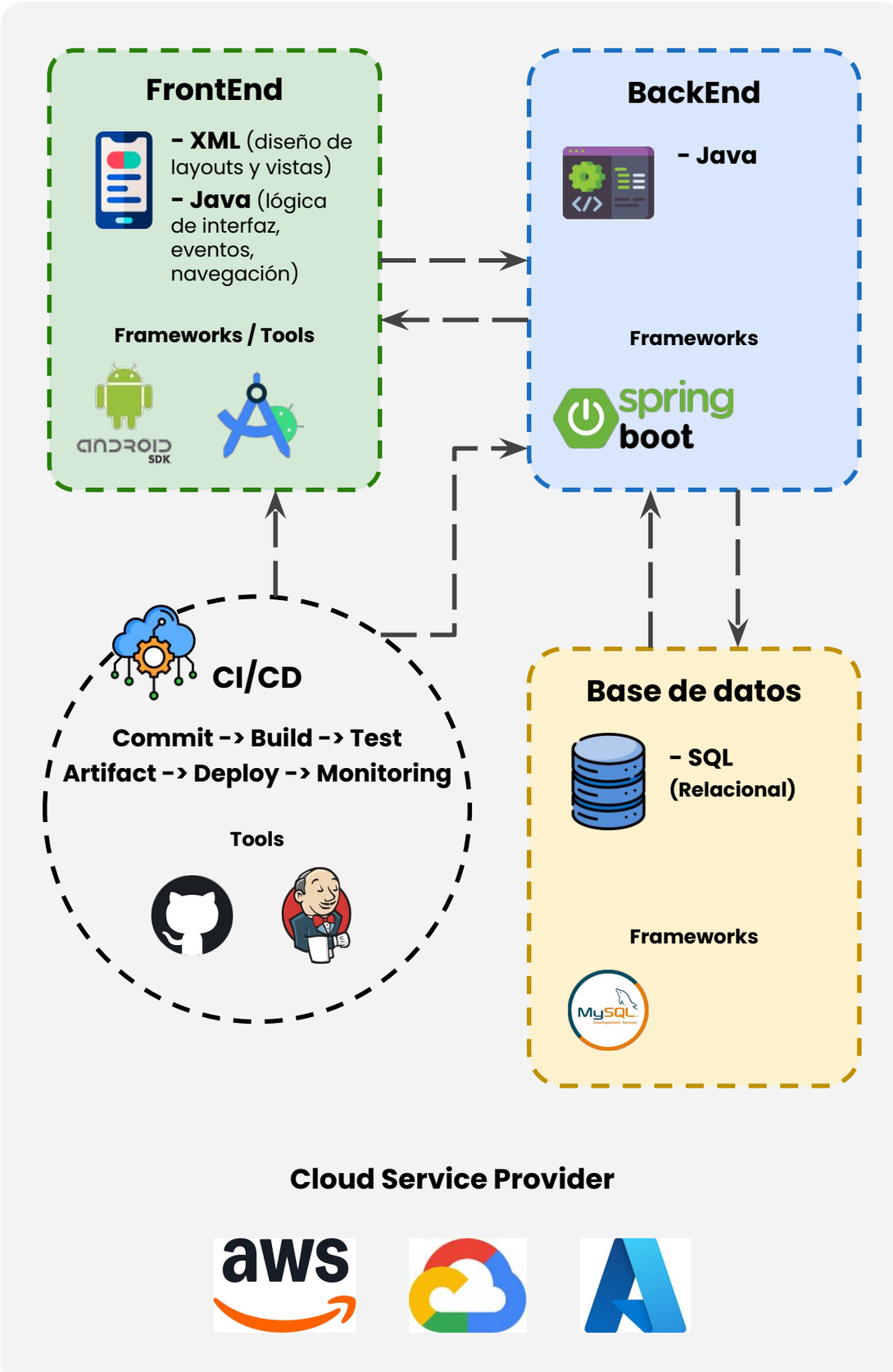
3. DECISIONES DE ARQUITECTURA (ADRs)

ADR-09: Seguridad: autenticación centralizada en el backend

- **Decisión:** Gestionar autenticación en un componente de seguridad del backend (por ejemplo, AuthController + SecurityConfig).
- **Contexto:** El sistema maneja credenciales y sesiones/tokens del cliente.
- **Alternativas:** Autenticación solo del lado cliente; credenciales sin cifrado.
- **Justificación:** Reduce riesgos y centraliza control de acceso.
- **Consecuencias:** Se deben definir políticas de expiración, protección de endpoints y almacenamiento seguro de credenciales.

4. STACK TECNOLÓGICO DEL SISTEMA

En esta sección se describen las tecnologías utilizadas para la implementación del sistema FoodExpress UQ, organizadas por capa: cliente móvil (frontend), backend y base de datos. Esta descripción es coherente con la arquitectura presentada en los diagramas C4 y corresponde al alcance definido para el rol del cliente.



VIGILADA MINEDUCACIÓN